# BEE 4750 Lab 2: Monte Carlo

**Name**: Evan Wu

**ID**: 5839048

### Due Date

Thursday, 10/02/25, 9:00pm

## Setup

The following code should go at the top of most Julia scripts; it will load the local package environment and install any needed packages. You will see this often and shouldn't need to touch it.

```
import Pkg
Pkg.activate(".")
Pkg.instantiate()
```

```
  Activating project at `c:\Users\evant\OneDrive\Documents\GitHub\lab-2-EvoTW-source\lab-2-E
Precompiling project...
   3751.1 ms    Distributions
   1031.3 ms    Distributions → DistributionsTestExt
  37532.8 ms    Plots
  3 dependencies successfully precompiled in 48 seconds. 188 already precompiled.
```

```
using Random # random number generation
using Distributions # probability distributions and interface
using Statistics # basic statistical functions, including mean
using Plots # plotting
```

## Overview

In this lab, we will conduct a Monte Carlo experiment and explore how sensitive Monte Carlo estimates are to the underlying probability distribution(s).

You should always start any computing with random numbers by setting a "seed," which controls the sequence of numbers which are generated (since these are not *really* random, just "pseudorandom"). In Julia, we do this with the `Random.seed!()` function.

```
Random.seed!(1)
```

```
TaskLocalRNG()
```

It doesn't matter what seed you set, though different seeds might result in slightly different values. But setting a seed means every time your notebook is run, the answer will be the same.

> **Seeds and Reproducing Solutions**
>
> If you don't re-run your code in the same order or if you re-run the same cell repeatedly, you will not get the same solution. If you're working on a specific problem, you might want to re-use `Random.seed()` near any block of code you want to re-evaluate repeatedly.

### Probability Distributions and Julia

Julia provides a common interface for probability distributions with the `Distributions.jl` package. The basic workflow for sampling from a distribution is:

1. Set up the distribution. The specific syntax depends on the distribution and what parameters are required, but the general call is the similar. For a normal distribution or a uniform distribution, the syntax is

   ```
   # you don't have to name this "normal_distribution"
   #   is the mean and   is the standard deviation
   normal_distribution = Normal( ,  )
   # a is the upper bound and b is the lower bound; these can be set to +Inf or -Inf for ar
   uniform_distribution = Uniform(a, b)
   ```

   There are lots of both univariate and multivariate distributions, as well as the ability to create your own, but we won't do anything too exotic here.

2. Draw samples. This uses the `rand()` command (which, when used without a distribution, just samples uniformly from the interval $[0, 1]$.) For example, to sample from our normal distribution above:

```
# draw n samples
rand(normal_distribution, n)
```

Putting this together, let's say that we wanted to simulate 100 six-sided dice rolls. We could use a Discrete Uniform distribution.

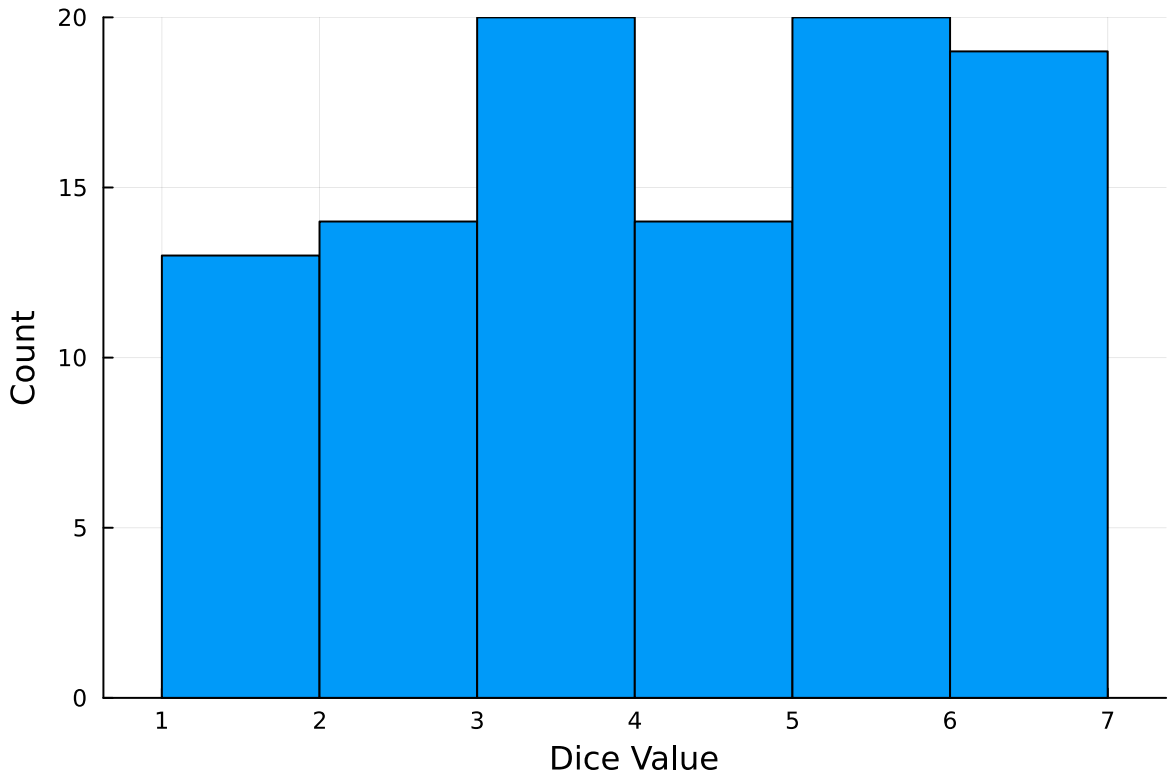
```
dice_dist = DiscreteUniform(1, 6) # can generate any integer between 1 and 6
dice_rolls = rand(dice_dist, 100) # simulate rolls
```

```
100-element Vector{Int64}:
 1
 3
 5
 4
 6
 2
 5
 5
 5
 2

 3
 6
 5
 5
 6
 3
 6
 6
 6
```

And then we can plot a histogram of these rolls:

```
histogram(dice_rolls, legend=:false, bins=6)
ylabel!("Count")
xlabel!("Dice Value")
```




### Instructions

**Remember to**:

- Evaluate all of your code cells, in order (using a `Run All` command). This will make sure all output is visible and that the code cells were evaluated in the correct order.
- Tag each of the problems when you submit to Gradescope; a 10% penalty will be deducted if this is not done.

## Problem

A common engineering problem is to quantify flood risk, which is typically computed by propagating a flood hazard distribution through a *depth-damage function* relating flood depths to economic damages. A reasonable depth-damage function for a house without a basement is a bounded logistic function,

$$d(h) = \mathbb{1}_{h>0} \frac{L}{1 + \exp(-k(h - h_0))},$$

where $d$ is the damage as a percent of total structure value, $h$ is the water depth (relative to the house's ground floor elevation) in m, $\mathbb{1}_{x>0}$ is the indicator function, $L$ is the maximum loss in USD, $k$ is the slope of the depth-damage relationship, and $h_0$ is the inflection point. We'll assume $L = \$200,000$, $k = 0.8$, and $h_0 = 3$.

For this problem, suppose that we have two different probability distributions characterizing annual maxima flood depths:

1. $h \sim LogNormal(1.2, 0.3)$;
2. $h \sim GeneralizedExtremeValue(3, 0.9, -0.15)$.

Plot histograms of samples from these distributions and conduct a Monte Carlo experiment for annual maximum flood damages using each. Answer the following questions:

- What are the Monte Carlo estimates of the expected value and the 99% quantile for the annual maximum damage the structure would suffer for each of these flood hazard distributions?
- How did you decide on your sample size?
- Why do you think the estimates differed or did not differ (you can also plot the depth-damage function to compare with the samples)?
- How might you decide (based on getting additional information, if needed) which distribution is "better"?

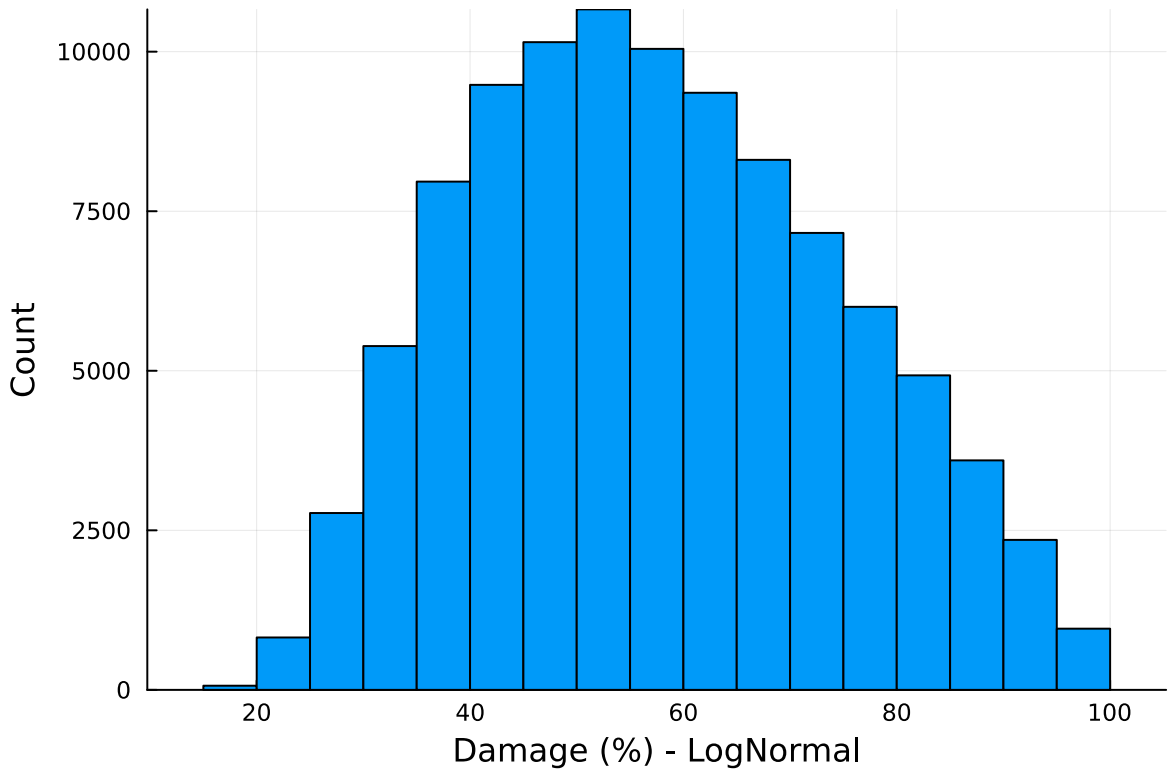
```
# Equation constants
L = 200000.0 # Money
k = 0.8 # Slope
h_0 = 3.0 # Inflection point
sample_size = 100000;
```

```
h_log = (LogNormal(1.2,0.3))
log_rolls = rand(h_log, sample_size)

dataset_log = []

for i in 1:sample_size
    dh = L / (1 + exp(-k * (log_rolls[i] .- h_0)))
    dh = dh / L * 100 # Converting into percentage
    push!(dataset_log, dh)
end

histogram(dataset_log, legend=:false, bins=30)
ylabel!("Count")
xlabel!("Damage (%) - LogNormal")
```




```
q_99 = quantile(dataset_log, 0.99)
q_99 = q_99 * L/100

q_50 = mean(dataset_log)
q_50 = q_50 * L/100

print("99th percentile damage (LogNormal): $q_99\n")
print("50th percentile damage (LogNormal): $q_50\n")
```

```
99th percentile damage (LogNormal): 189728.6638254642
50th percentile damage (LogNormal): 115070.72274528582
```

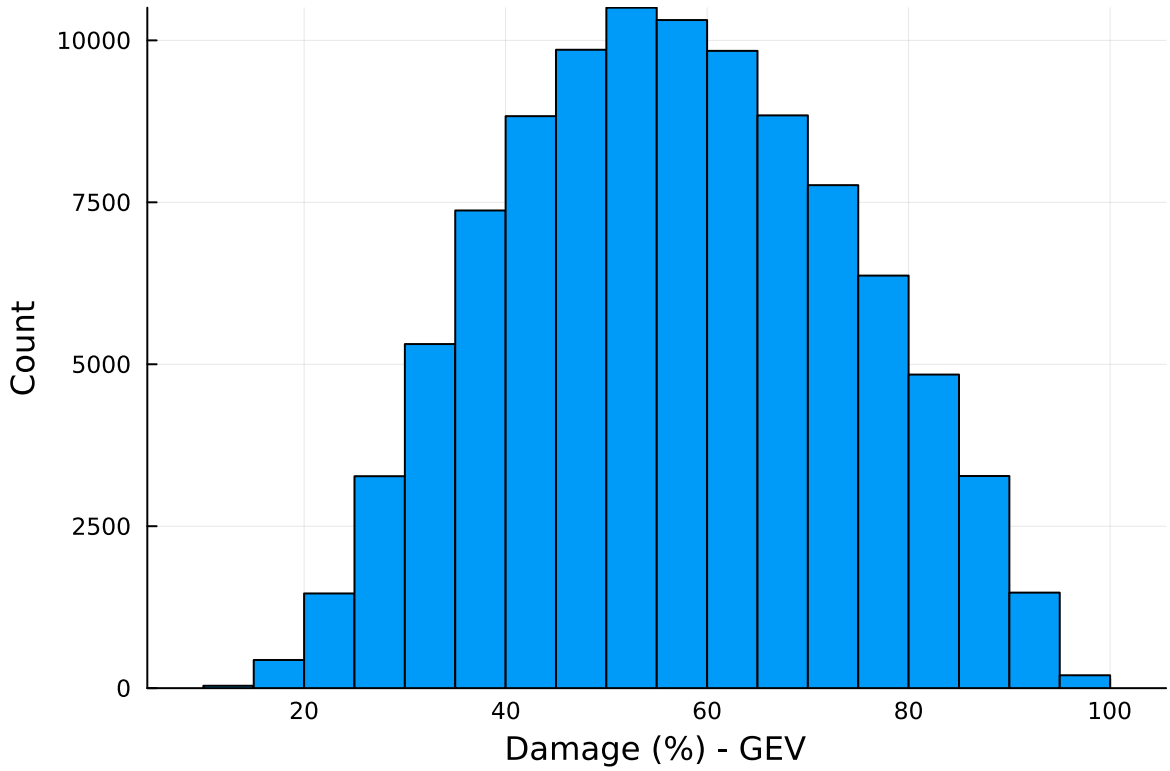
```
h_GEV = GeneralizedExtremeValue(3, 0.9, -0.15)
GEV_rolls = rand(h_GEV, sample_size)

dataset_GEV = []

for j in 1:sample_size
    if GEV_rolls[j] < 0
```

```
        dh = 0
        push!(dataset_GEV, dh)
    else
        dh = L / (1 + exp(-k * (GEV_rolls[j] .- h_0)))
        dh = dh / L * 100 # Converting into percentage
        push!(dataset_GEV, dh)
    end
end

histogram(dataset_GEV, legend=:false, bins=30)
ylabel!("Count")
xlabel!("Damage (%) - GEV")
```




```
q_99 = quantile(dataset_GEV, 0.99)
q_99 = q_99 * L/100

q_50 = mean(dataset_GEV)
q_50 = q_50 * L/100
```

```
print("99th percentile damage (GEV): $q_99\n")
print("50th percentile damage (GEV): $q_50\n")
```

```
99th percentile damage (GEV): 183424.03036198046
50th percentile damage (GEV): 113566.45780502993
```

What are the Monte Carlo estimates of the expected value and the 99% quantile for the annual maximum damage the structure would suffer for each of these flood hazard distributions? - The average cost of damage with the LogNormal distribution would be 115070 dollars - The 99th quantile for annual maximum damage with the LogNormal distribution would be 189728 dollars - The average cost of damage with the GEV distribution would be 113566 dollars - The 99th quantile for annual maximum damage with the GEV distribution would be 183424 dollars

How did you decide on your sample size?

Sample size of 100,000 was chosen to ensure consistent patterns and differences between the histograms generated by the LogNormal and GEV distributions can be spotted. Separate histograms were created starting from 1000 to 1000000, increasing each subsequent number by a factor of 10. - At n = 1000, no consistent pattern was deducible from the data constructed - At n = 10000, a distribution curve was formed but it was hard to find a consistent mean value since each histogram had different midpoints. Additionally, the edges of the curve were random due to low sample counts. - At n = 100000, the curve has a consistent distribution and the edges do not change significantly. The mean and 99th quantile have minimal changes. - At n = 1000000, it is visually indistinguishable as when n = 100000, except it required additional computational time, therefore it is unnecessary to run additional samples larger than 100000.

Why do you think the estimates differed?

The differences come from how the random distributions are generated. The LogNormal distribution works around generating values where their logarithmic values are on a normal distribution. On the other hand, the GEV distribution focuses on distributing based on maximums.

The significance difference is that the LogNormal distribution only models positives values whereas the GEV distribution can model negative numbers. This is important because it means every sample will be larger than 0 and every damage instance will be computed, while the GEV may produce negatives that are not considered. This skews the mean of the LogNormal distribution to the right in comparison to the mean of the GEV distribution.

How might you decide which distribution is better?

Deciding which distribution is preferred will depend on what information is desired. If looking for average values and distributions, the Log Normal distribution would be preferred. This

is because the LogNormal distribution has a larger concentration of samples near the mean value.

If considering the extreme scenarios and how much damages would cost at high damage percents, the GEV distribution should be used instead. This is since the GEV distribution has more samples on the tail ends of the curve, so the extreme cases would be more accurate.

## References

Put any consulted sources here, including classmates you worked with/who helped you.